A2: Architecture

A larger program is easier to build when each part has one clear job. The hard part is often deciding what those parts should be before any of them exist. This guide gives you a reasonable default architecture for Past Performance. You may follow it exactly or choose another organization that satisfies R4.

The flow of information

The completed program has one main path:

```
command-line input
        |
        v
market data + selected strategy + initial cash
        |
        v
backtest engine <----> strategy
        |
        v
simulation result
        |
        v
final summary + date/value CSV
```

The **backtest engine** is the coordinator. It advances through the market data one date at a time, shows the strategy the new closing price information available on that date, receives the strategy's proposed orders, processes them, and records the resulting portfolio state.

Keeping that flow in mind helps answer a useful design question: **which unit should own each decision?** A decision should normally have one owner rather than being distributed in several places.

Compilation units

A compilation unit is one `.ml` file with an optional `.mli` interface file. The following organization is a reasonable design:

Compilation unit	What it owns	What it does not own
`Market`	Opening and reading valid CSV files of market data; dates, security names, and prices	Trading decisions and portfolio updates

Compilation unit	What it owns	What it does not own
`Money`	Parsing and formatting integer cents	Market data and portfolio updates
`Order`	The representation of proposed buys and sells and their outcomes	Deciding which orders a strategy should propose
`Portfolio`	Cash, holdings, order application, rejection, and valuation	Moving through input dates or formatting reports
`Strategy`	The common strategy module type and the information exchanged with the engine	Any one strategy's private rule or state representation
Strategy implementations	Cash, buy-and-hold, and the stateful rule	Executing orders or receiving price rows other than the current one
`Backtest`	The date-by-date simulation and its recorded result	Strategy-specific decisions and output formatting
`Report`	Formatting the complete human-readable summary as text and writing the date/value CSV	Recalculating trades or portfolio values
`Command`	Command-line parsing	Accounting rules and strategy algorithms
`Main`	The entry point, orchestration, and terminal output	Any of the above responsibilities

Strategy implementations could be separate compilation units or nested modules collected in one compilation unit.

Interfaces determine boundaries

An interface is both a **promise to clients** and **freedom for the implementer**. Clients may rely on the promised operations; the implementation may change everything else without forcing those clients to

change.

For example, clients of a portfolio unit need ways to obtain cash and holdings, apply an order, and calculate value. They need not depend on whether holdings are stored in a list, map, or some other structure. Hiding that choice therefore can be helpful.

Expose operations that another unit or an OUnit test genuinely needs. Avoid exposing a helper merely because it exists or you want to test it — the test suite should concentrate on stable functionality without needing to access changeable internal implementation details. At the same time, do not try to hide the greatest possible number of functions: an interface that makes normal use (or focused testing) awkward is not automatically better.

Focus your design on strategies

The strategy modules are the most important part of the design of your application. Think of a strategy as having this lifecycle:

1. The strategy is initialized with general facts it may know, such as the security names, initial cash, and supplied option values.
2. On each date, the engine gives the strategy an observation containing only the new information available that date; if the strategy needs earlier prices, it must store them in its private state.
3. The strategy returns its proposed orders and the state it will carry to the next date.

As a suggested starting point, here is an interface design that supports the lifecycle described above:

```
type init_context = {
  securities : string list;
  initial_cash : int;  (* integer cents *)
  options : (string * string) list;
}

type observation = {
  date : string;
  prices : (string * int) list;  (* current date only *)
  cash : int;
  holdings : (string * int) list;
}

module type S = sig
  type state

  val name : string
  val init : init_context -> (state, string) result
  val step : state -> observation -> Order.t list * state
end
```

The shape of `observation` prevents a strategy from using the interface to request arbitrary price rows. A type cannot establish where a particular observation value came from, so the engine must also ensure that each call to `step` receives the row for the date currently being processed.

Each part has a purpose:

- `init_context` contains facts known before the first date, including any recognized option values. It contains no price rows.
- `init` interprets and validates the supplied options and constructs the initial state.
- `observation` contains the current date's prices and portfolio view. A strategy that needs earlier prices stores them in its private state as dates are processed.
- `step` returns a list of orders to be processed in the sequence proposed. It returns the next state; it does not mutate the state.
- The declaration `type state` makes that type abstract. Each strategy can use a different representation without exposing it to the engine.

You are welcome to adapt the suggested code above, but avoid these two tempting changes which would break the intended encapsulation boundary:

1. Do not add price rows to `init_context`, or any earlier or later row to `observation`.
2. Do not replace the abstract `state` with an exposed type containing every strategy's private information.

Designing the terminal and file I/O responsibilities

In this suggested architecture, the filesystem operations are library capabilities rather than responsibilities of the terminal interface. `Market.load` opens and reads the market file, `Report.write` writes the equity CSV, and `Report.render` produces the complete human-readable summary as one string. That string includes every label, formatted value, line break, and other display choice. `Main` coordinates the operations and reports errors; it prints the output summary.

The backtest engine, portfolio, and strategy modules perform no file or terminal I/O. This division would allow the same data-loading, simulation, and reporting library to survive if a graphical interface later replaced the terminal interface. A graphical interface could use the structured backtest result directly and simply ignore the text-rendering operation.

Warning signs of a bad design

Watch out for these warning signs that a design is not well-factored:

- The engine branches on a strategy name or implements part of a strategy's trading rule.
- A strategy receives any price row other than the current date's row.
- Another compilation unit constructs, inspects, or modifies a strategy's private state.
- Command-line or reporting code applies trades or independently recomputes portfolio values.
- The same parsing, accounting, or validation rule is implemented in multiple units and can drift out of agreement.

- Two compilation units depend on each other, perhaps because their responsibilities are not clearly defined.